Deep Learning: clasificador de canciones

Santiago Alducin Villaseñor, *A01707122*
05-11-2026

***Abstract*—Utilizando de base el Artículo de "*Musical genre classification using advanced audio analysis and deep learning techniques*" de Ahmed et.al. (2024) trato de incorporar 2 arquitecturas CNN para la clasificación de género musical. Pese a copiar las arquitecturas explicadas en la fuente original, los resultados obtenidos se encuentran entre un 10 y 20% por debajo de los esperados.**

INTRODUCTION

Escuchar música es una de las actividades que más hago, en transporte, mientras leo o en lo que realizo otras actividades. Pero no soy bueno identificando el género de muchas canciones que me gustan, o que me gustaría escuchar una canción con un estilo muy similar a otra que me guste, pero me cuesta encontrar esa relación. Por ello se me ocurrió que este proyecto tenga algo relacionado con canciones.

El objetivo general del proyecto es lograr crear un clasificador de género musical. En este definí los criterios de éxito.

Criterios de éxito

1. Identificar correctamente los fragmentos de una canción del dataset con su género (Validación y Pruebas)
2. Poder identificar el género de la canción completa

Estoy utilizando de base la metodología explicada por Ahmed, et.al. en "Musical genre classification using advanced audio analysis and deep learning techniques" (2024). Este es un estudio del estado del arte donde compara diferentes modelos y acercamientos a este problema por diferentes personas, y compara sus resultados. Lo estoy usando como guía para fundamentar y guiar el proyecto porque explica paso a paso las diferentes estrategias y da la teoría y matemáticas detrás para el análisis.

MARCO TEÓRICO

El artículo anteriormente mencionado aborda la teoría y diferentes estrategias para la clasificación de música, citando a diferentes autores con sus estrategias para este problema, y resultados. Entre ellos explica arquitecturas de Feed forward neural network (FNN), modelos de convolución (CNN), modelos de convolución modificados (Modified CNN), Recurrent Neural Network (RNN-LSTM), Support Vector Machine (SVM), K Nearest Neighbors (KNN) y compara sus resultados.

En esta muestra que la red modifica CNN fue la que mejor resultados tuvo con una exactitud del 92.7%, seguida de la CNN regular con 85.56% y en último lugar KNN con 70%.

El proceso que sigue el artículo, y que uso como guía para este problema es la extracción de fragmentos de 30 segundos de una canción. Transformar la señal en Mel-Frequency Cepstral Coefficients (MFCC) y almacenarlos para luego procesarlos con los diferentes modelos y hacer el diagnóstico final. Se usa este modelo de frecuencia para simular la audición humana, extrayendo señales importantes del audio y es menos afectada por el ruido.

MFCC tiene varios puntos positivos, elimina el ruido de la señal facilitando extraer características importantes del género musical. Al igual que mejora el rendimiento del modelo y acelerando el tiempo de procesamiento al disminuir la complejidad computacional.

Decidí utilizar los modelos de CNN y CNN modificada, debido a que son temas relacionados con lo visto en clase y aquellos que demostraron mejores resultados entre todas las opciones. Primero implementando CNN y CNN modificada, tras el diagnóstico ver qué resultados muestra y decidir cual modelo tratar de mejorar.

DATOS

Se van a utilizar el dataset GTZAN (Tensorflow, s.f.), que recopila fragmentos de 30 segundos de 100 canciones por género como fuente de datos inicial.

GTZAN es una colección de diez géneros musicales, con 100 canciones por género en formato WAV que duran 30 segundos. No pude encontrar la importación original de Tensorflow, pero si los archivos locales, así que estos los leo manualmente e integro en un dataframe de pandas.

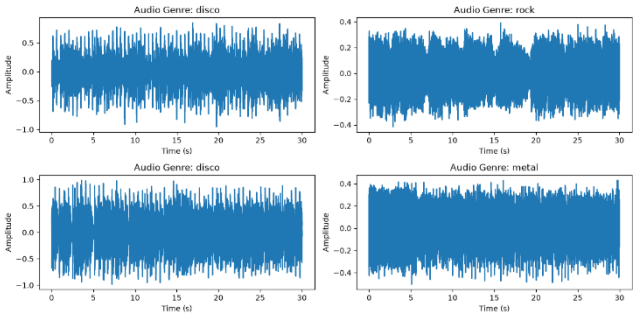
En total son 999 canciones, debido a que hay una canción de Jazz que está mal guardada y da error al leerla.

Columna	Formato
Audio	WAV (Librosa)
Genre	String
File	Path

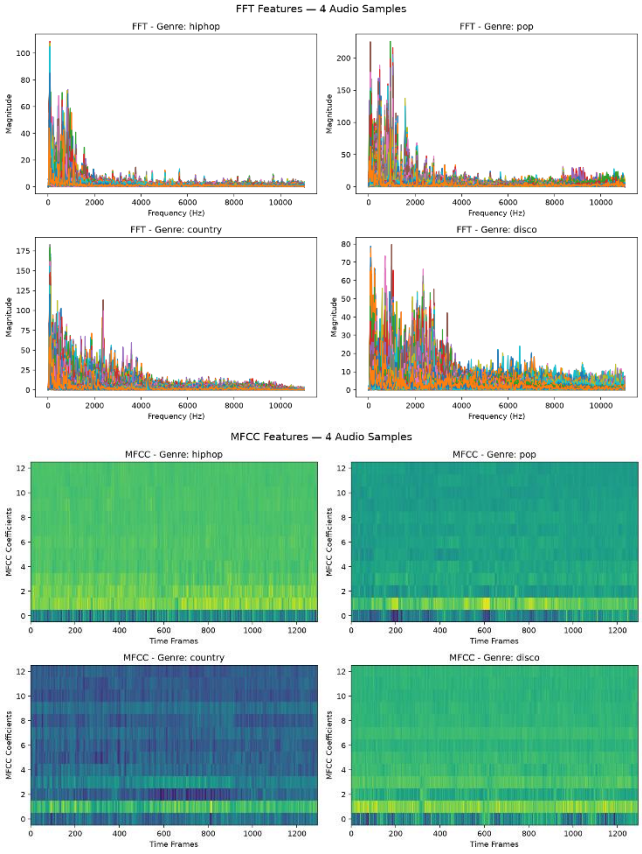
ENTENDIMIENTO DE LOS DATOS

Para la primera fase de entendimiento de los datos cargué los diferentes archivos de GZTAN en Python generando el dataframe antes mencionado. Una vez que estaba cargado hice una gráfica para mostrar: Las frecuencias de audio de la canción, su transformación de Fourier y MFCC. Este último siendo el que utilizará el modelo.

Primero con las frecuencias de audio, no hay muchos detalles que se puedan apreciar, la música disco tiene algunos picos ligeros, la música metálica parece ser la más estruendosa, y el rock un poco por detrás del metal, con mayor densidad.



En las gráficas de Transformadas de Fourier un poco similar se puede ver que la música pop es aquella con mayor magnitud, e igual están las frecuencias de Mel. Honestamente no sabría cómo interpretar las gráficas, pero el propósito del modelo es poder extraer las características más importantes de estas para la clasificación de las canciones.



El dataset contiene 100 canciones por género, siendo los géneros: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae. De estos, casi todos tienen 100 canciones excepto Jazz, debido a que una de sus canciones es corrupta.

Todas las canciones duran 30 segundos.

PREPARACIÓN DE LOS DATOS

Para el funcionamiento correcto del modelo es necesario transformar las canciones de su forma original (archivos .wav), en su Mel-Frequency. Para ello encontré algunos problemas cargando toda la información y transformándola en su totalidad debido a que mi computadora, pese a contar con 16 Gbs de RAM, se quedaba sin memoria. Por ello opté por separar los archivos en batches de 50 canciones y guardarlas en sus propios archivos npz.

Cargo la información, proceso el audio para que tenga el tamaño adecuado dependiendo de la frecuencia (22050) y la duración de 30 segundos. Luego se extraen MFCC y FFT, este último no será utilizado en esta primera versión del modelo, pero consideré que sería bueno tenerlo de una vez a la mano para una futura mejora.

Se almacenan la información de 50 canciones antes de guardarlo en un archivo .npz, generando un total de 20 archivos que se utilizarán para el modelado.

FUNCIONAMIENTO DEL MODELO

El modelo, como se mencionó anteriormente sería una red neuronal Convolucional. Debido al mismo problema de falta de memoria para trabajar con los datos de manera directa y bruta igual trabajé con batches. En este momento cargué el nombre de los archivos para un Split de 70% entrenamiento, 15% validación y otro 15% pruebas.

Luego decidí ejecutar una normalización y encoder en los datos para el entrenamiento, para esto cargué todos los datos uno a la vez para almacenar los géneros para que pueda codificarlos, y guardar información de las MFCC. Esto me ayudo a tener la información general de todos los archivos para tener los labels codificados para el entrenamiento y pruebas al igual que tener la media, y desviación estándar para la normalización de las frecuencias.
Esta normalización la guardé igualmente en un archivo para su uso posterior para el archivo de pruebas.

Ahora, para realmente el entrenamiento: creé un generador que carga las frecuencias MFCC y género de las canciones en batches de 16 para el proceso de entrenamiento, validación y pruebas.

El modelo, empieza por un input con el tamaño de entrada (MFCC) y de salida (Géneros), seguido de 3 capas de Convolusión2D y MaxPooling que progresivamente van extrayendo la información de las imágenes. Finalmente se encuentra una capa densa, un Dropout para evitar overfitting y una última capa densa que hace la selección correcta utilizando softmax.

Al compilar el modelo se utiliza Adam para optimizar el modelo, CategoricalCrossentropy para poder ajustar el error y las métricas de exactitud y precisión, y F1 esto para poder tener una mejor idea de cómo se está adaptando el modelo y tomar en cuenta en realidad cuantas veces reconoce correctamente el género de una canción.

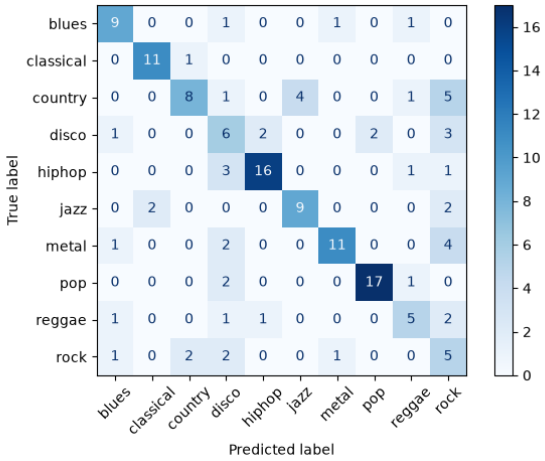
Esta versión es una ya un poco modificada de la versión original del modelo que menciona el artículo debido a que en su momento le entendí mal, pero los resultados no fueron terribles.

RESULTADOS INICIALES

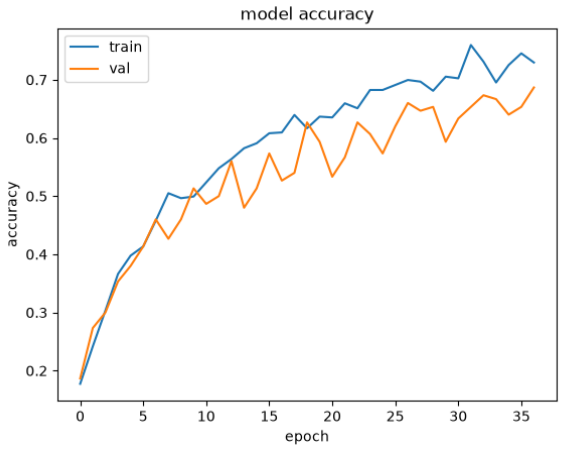
Métrica	Valor
Pérdida	1.0848
Exactitud	64.67%
F1	64.11%
Precisión	74.78%

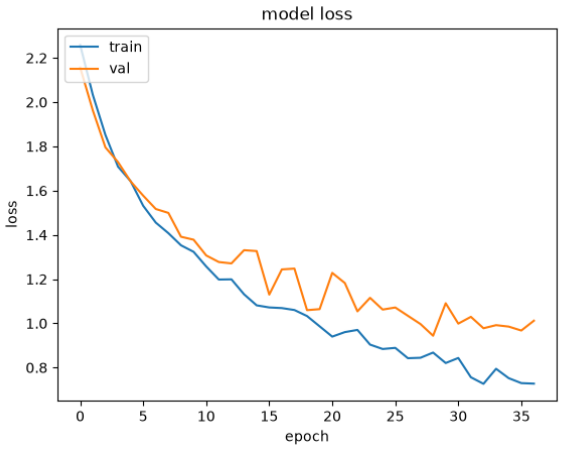
Reporte de clasificación

Género	precision	recall	f1-score	Cantidad
blues	0.69	0.75	0.72	12
Classical	0.85	0.92	0.88	12
Country	0.73	0.42	0.53	19
Disco	0.33	0.43	0.38	14
Hiphop	0.84	0.76	0.80	21
Jazz	0.69	0.69	0.69	13
Metal	0.85	0.61	0.71	18
Pop	0.89	0.85	0.87	20
Reggae	0.56	0.50	0.53	10
Rock	0.23	0.45	0.30	11



En el modelo se puede ver que la precisión de este se encuentra alrededor de 60-70%. Hay una gran cantidad de error en los géneros de Rock, Disco y Reggae, mientras que Clásico, Pop e Hip-hop son aquellos que tienen un menor error.



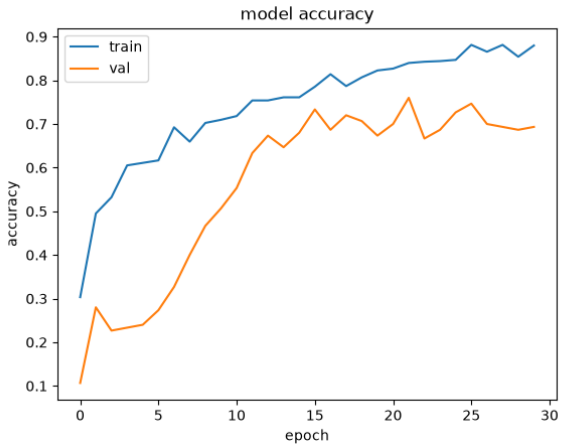


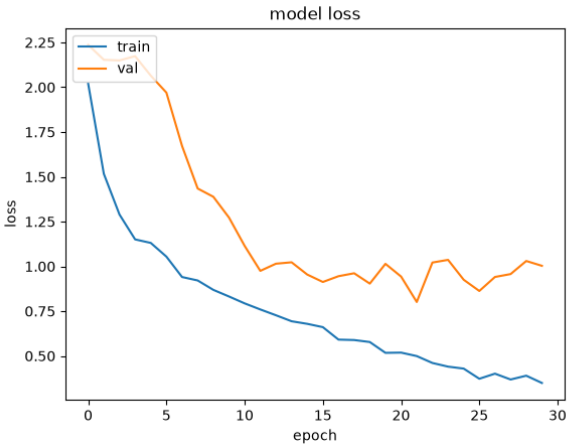
DIAGNÓSTICO

Las métricas utilizadas para esta primera iteración del proyecto muestran resultados por debajo de lo esperado y reportado en el reporte original de Ahmed, et.al (2024). Esto puede deberse a una variabilidad en la construcción del modelo dado que la construcción de este fue una interpretación de la explicación utilizada al igual que la ausencia de hiperparámetros reportados en el reporte.

	Resultados del modelo	Resultados del reporte
Accuracy	69.33%	85.56%
F1	0.6788	0.85
Precission	0.7750	0.89

Dos cosas que puedo identificar es que el reporte original utiliza 100 ciclos de entrenamiento, pero mi modelo se utiliza early stopping para agilizar el entrenamiento, deteniendoce después de 8 iteraciones sin un cambio significativo en la pérdiad, por lo que ni siquiera llega a los establecidos 50 ciclos.



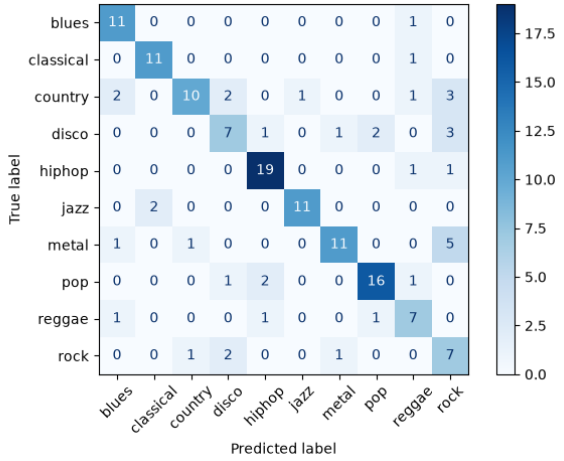


MEJORA CNN

Traté de adaptar un poco más la arquitectura de mi modelo a la explicada en el artículo, utilizando la descripción tal cual se explica en el artículo no me dio muy buenos resultados, pero si utilicé estrategias que menciona. En este caso cambié el orden de las capas de convolución para que vayan disminuyendo en lugar de aumentando de tamaño (128-64-32), al igual que integrar un BatchNormalization después de las primeras dos capas.

	Resultados del modelo Original	Mejora	Resultados del reporte
Accuracy	69.33%	73.33%	85.56%
F1	0.6788	0.7255	0.85
Precision	0.7750	0.8077	0.89

Esta versión aún no alcanza los resultados obtenidos del reporte, pero tiene un aumento rescatable en comparación a la primera versión.

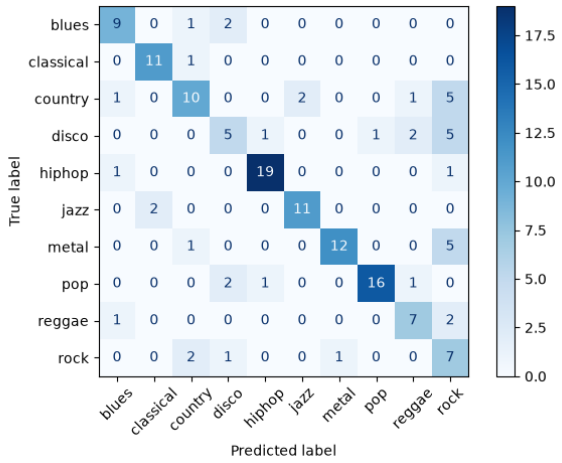


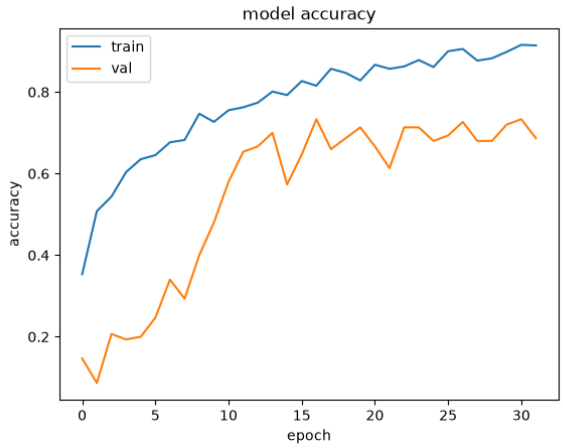
IMPLEMENTACIÓN CNN MODIFICADA

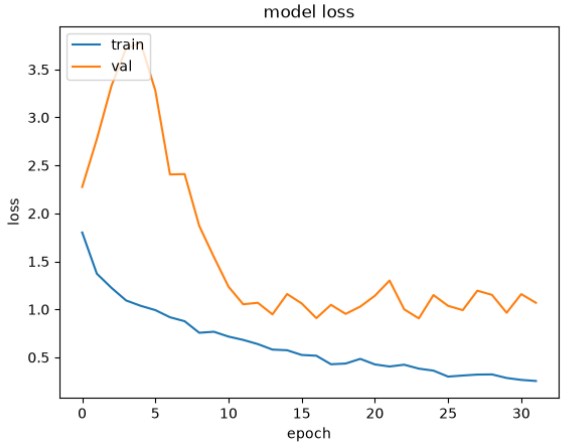
	CNN Modificada	Resultados del reporte
Accuracy	71.33%	92.7%
F1	0.708	0.96
Precision	0.7	0.93

La CNN Modificada, pese a tratar de crear la arquitectura en base a lo explicado en el artículo original los resultados fueron por debajo de lo esperado.

Los resultados demuestran que las métricas están por encima del modelo CNN original que implementé, pero se encuentra un poco por debajo de la versión mejorada. Pese a esto, ninguno de los modelos ha obtenido resultados comparables con lo reportado en el artículo, por lo que los hiperparámetros y arquitectura siguen sin llegar a los resultados esperados.







CONCLUSIONES

Durante el desarrollo del proyecto implementé 3versiones de redes convolucionales para determinar la categoría de una canción. Los resultados no fueron los esperados, ya que el rendimiento del modelo es inferior a lo esperado de acuerdo con la teoría y el experimento de Ahmed, et.al. (2024), pero si obtuvo resultados con un F1 Score arriba del 50% en todas las pruebas.

La primera arquitectura es una red convolucional que va aumentando de dimensiones de 32 a 128 con capas de MaxPooling entre ellas conectada a unas dos capas densas para determinar la categoría. Esta fue producto de una confusión personal al momento de establecer el tamaño de las capas, pero tuvo un rendimiento mejor de lo esperado para la primera iteración.

La segunda iteración sigue la misma estructura base, pero con las capas convolucionales disminuyendo las dimensiones de 128 a 32, y con BatchNormalization. Este es el que mejor rendimiento tuvo de las 3 arquitecturas.

El tercero era la red convolucional modificada, que usa dimensiones convolucionales de 256 a 64, e igual utiliza BatchNormalization, aunque sorprendentemente tuvo peores resultados que la versión anterior, mostrando que más dimensiones no es siempre lo ideal.

Finalmente, cabe mencionar que todos los modelos muestran un grado de overfit, la primera iteración este overfit era pequeño y no tan importante, pero las siguientes versiones parece que las pruebas tuvieron resultados mucho más ruidosos, y con una diferencia de validación y entrenamiento mayor en comparación con la v1.

REFERENCIAS

Ahmed, M., Rozario, U., Kabir, M. M., Aung, Z., Shin, J., & Mridha, M. F. (2024). Musical genre classification using advanced audio analysis and deep learning techniques. IEEE Open Journal of the Computer Society, 5, 457–467. https://doi.org/10.1109/OJCS.2024.3431229 . Recuperado de: https://ieeexplore.ieee.org/abstract/document/10605044

TensorFlow. (s. f.). GTZAN: A dataset for music genre classification. TensorFlow Datasets. Recuperado el 18 de octubre de 2026, de https://www.tensorflow.org/datasets/catalog/gtzan

Patel, S. J. (2026). Sunnify Spotify Downloader (Versión 2.5.0) [Software de computación]. GitHub. https://github.com/sunnypatell/sunnify-spotify-downloader